EC3290 – Software Requirements Engineering

Topic 1 – Introduction to Requirements Engineering

CLO 1

4 Hours

Introduction to Software Engineering

Definition:
Software engineering is the application of engineering principles to the development, operation, and maintenance of software systems. It involves a systematic, disciplined, and quantifiable approach to software development, ensuring high quality and efficiency.

Key Components:

1. **Processes:**
   These are structured sequences of activities that guide software development, such as planning, analysis, design, implementation, testing, and maintenance. Processes ensure a systematic approach and are often tailored to project needs.

2. **Tools:**
   Software tools are used to support processes and methodologies. Examples include integrated development environments (IDEs), version control systems, and testing frameworks.

3. **Methodologies:**
   These are frameworks that dictate how processes are implemented. Examples include Agile, Waterfall, and DevOps. Methodologies help teams align on goals and strategies throughout the project lifecycle.

Software Development Lifecycle (SDLC) Overview

Planning

Analysis

Design

Implementation

Maintenance

Importance of Processes in Software Development

- **Ensures Consistency and Quality:**
  Well-defined processes help maintain consistency across teams and projects. By following standard procedures, teams can deliver software that meets quality expectations and adheres to best practices.
- **Manages Complexity and Reduces Risks:**
  Software development often involves handling complex systems and technologies. Processes help break down these complexities into manageable steps and identify potential risks early, allowing teams to mitigate them proactively.
- **Facilitates Communication Among Stakeholders:**
  Processes provide a common framework and terminology for developers, project managers, clients, and other stakeholders. This fosters better collaboration, clearer expectations, and alignment on project goals.

Relationship Between Software Processes and Requirements

- **Foundation of Subsequent Phases:**
  Requirements serve as the cornerstone of software development. They define what the software must do and set the scope of the project. All subsequent phases—design, implementation, testing, and maintenance—rely on accurate and comprehensive requirements.
- **Impact of Poor Requirements:**
  Ambiguous, incomplete, or incorrect requirements can lead to misaligned development efforts, increased costs, and delays. In worst-case scenarios, poor requirements can result in the complete failure of a project.
- **Processes to Improve Requirement Quality:**
  - **Requirement Elicitation:** Engaging stakeholders through interviews, surveys, and workshops to gather detailed requirements.
  - **Requirement Validation:** Ensuring requirements are clear, consistent, and feasible before moving to the design phase.
  - **Requirement Management:** Continuously tracking and updating requirements throughout the project lifecycle.

Definition of Requirements Engineering

- **Definition:**
  Requirements engineering is the systematic process of identifying, documenting, and maintaining software requirements. It ensures that the software being developed aligns with user needs and business goals. This process is iterative and involves continuous interaction with stakeholders.
- **Key Activities in Requirements Engineering:**
  1. **Elicitation:**
     Gathering requirements from stakeholders through interviews, surveys, observation, and workshops.
  2. **Documentation:**
     Structuring the gathered requirements into clear, unambiguous, and verifiable documents, such as Software Requirement Specifications (SRS).
  3. **Validation:**
     Ensuring that requirements are complete, feasible, and correctly understood by all parties.
  4. **Management:**
     Maintaining and updating requirements throughout the software development lifecycle to address changes and evolving needs.

Role of Requirements Engineering

- **Bridges the Gap Between Stakeholders and Developers:**
  Requirements engineering acts as a mediator to translate stakeholder needs into technical specifications. It ensures that developers understand what the stakeholders want and why it's important.
- **Establishes a Clear Understanding of Project Goals:**
  By defining and documenting requirements upfront, all project participants—stakeholders, developers, testers, and managers—gain a shared understanding of the project's objectives and scope.
- **Supports Decision-Making:**
  Well-defined requirements provide a basis for making informed decisions during the project, such as prioritizing features or resolving conflicts between stakeholder expectations.
- **Enhances Collaboration:**
  It creates a structured process for continuous communication between the technical and non-technical members of the project team.

Importance of Requirements Engineering

- **Prevents Scope Creep:**
  Requirements engineering helps in clearly defining the project scope, preventing uncontrolled expansion of features or functionality during development. This ensures the project stays on track and within budget.
- **Reduces Development Costs and Time:**
  Addressing and refining requirements early in the project lifecycle helps identify potential issues and misunderstandings, reducing costly rework during later stages.
- **Improves Software Quality and User Satisfaction:**
  By focusing on user needs and expectations, requirements engineering ensures that the delivered software is both functional and user-friendly, leading to higher satisfaction and usability.
- **Minimizes Risks:**
  A clear understanding of requirements helps identify risks early, such as technical feasibility or conflicts between stakeholder priorities, enabling better risk management strategies.
- **Ensures Compliance:**
  For industries with regulatory requirements, the process ensures that all necessary standards are met.

Challenges in Requirements Engineering

- **Changing Requirements:**
  Stakeholders' needs may evolve over time due to market changes, new business goals, or feedback from early stages of development. Managing these changes while minimizing their impact is a significant challenge.
- **Ambiguities and Misunderstandings:**
  Requirements may be written or communicated in a way that is unclear, leading to differing interpretations by stakeholders and developers.
- **Communication Gaps Among Stakeholders:**
  Stakeholders may have conflicting goals or use different terminology, making it challenging to align their expectations.
- **Limited Stakeholder Involvement:**
  If stakeholders are not actively engaged in the requirements process, critical needs may be overlooked or misunderstood.
- **Technical Feasibility Issues:**
  Some requirements may not be technically or financially feasible, requiring negotiation and adjustment to align with constraints.

Challenges in Requirements Engineering

- **Addressing These Challenges:**
  - Use clear, unambiguous language and visual aids like diagrams.
  - Involve stakeholders continuously throughout the process.
  - Employ requirement management tools for tracking changes.
  - Conduct regular validation sessions to ensure alignment among all parties.

Overview of Requirements Engineering Activities

- **Requirements Engineering Activities:**
  1. **Elicitation:**
     The process of gathering requirements from stakeholders to understand what the system should do.
  2. **Analysis:**
     Understanding, refining, and prioritizing requirements to ensure they are feasible and align with project goals.
  3. **Specification:**
     Documenting requirements in a structured format to serve as a reference for all project phases.
  4. **Validation:**
     Verifying that the documented requirements meet user needs and are complete, consistent, and feasible.
  5. **Management:**
     Handling changes to requirements throughout the project lifecycle to ensure alignment with evolving needs.

Elicitation

- **Definition:**
  Elicitation is the process of gathering and discovering requirements from stakeholders and other sources. It is the foundation of requirements engineering and involves active engagement with stakeholders.
- **Techniques:**
  1. **Interviews:**
     Conducting one-on-one or group discussions to understand stakeholder needs and expectations.
  2. **Surveys and Questionnaires:**
     Distributing structured forms to collect information from a larger group of stakeholders.
  3. **Brainstorming Sessions:**
     Facilitating collaborative sessions to generate ideas and uncover hidden needs.
  4. **Document Analysis:**
     Reviewing existing documentation, such as business plans or system manuals, to extract relevant requirements.

Analysis

- **Definition:**
  Analysis involves understanding, refining, and organizing elicited requirements to ensure they are feasible, consistent, and aligned with project goals.
- **Key Activities:**
  1. **Identifying Dependencies:**
     Understanding how different requirements are related and ensuring no conflicts exist between them.
  2. **Prioritizing Requirements:**
     Categorizing requirements based on their importance and urgency to stakeholders. Techniques like the MoSCoW method (Must have, Should have, Could have, Won't have) are often used.
  3. **Feasibility Assessment:**
     Ensuring that the requirements can be achieved within the project's technical, financial, and time constraints.

MoSCoW Method – Must Have

- **Definition:** These are the requirements that are critical to the success of the project. Without them, the system cannot function, and the project would be considered a failure.
- **Characteristics:**
  - Essential for achieving business goals or meeting legal and regulatory requirements.
  - Directly impacts the system's core functionality.
- **Examples:**
  - A banking app **must have** secure login functionality.
  - An e-commerce website **must have** a shopping cart feature.

MoSCoW Method – Should Have

- **Definition:** These are important requirements but not critical for immediate success. They are often considered high-priority but not mandatory for the first release.
- **Characteristics:**
  - Adds significant value but can be deferred if necessary.
  - May be included if time and resources permit.
- **Examples:**
  - A system **should have** multiple language support.
  - A project management tool **should have** integration with other tools like Slack or Teams.

MoSCoW Method – Could Have

- **Definition:** These are desirable but not essential requirements. They have a lower priority and are often included if time, budget, and resources allow.
- **Characteristics:**
  - Nice-to-have features that enhance user experience.
  - Does not impact core functionality or business goals.
- **Examples:**
  - A mobile app **could have** a dark mode.
  - A website **could have** animated transitions between pages.

MoSCoW Method – Won’t Have

- **Definition:** These are requirements that are agreed to be out of scope for the current project or release but may be considered for future iterations.
- **Characteristics:**
  - Explicitly documented to avoid confusion or scope creep.
  - Can be revisited in later phases or versions.
- **Examples:**
  - A system **won’t have** voice recognition in the initial release.
  - A website **won’t have** augmented reality features in Phase 1.

Specification

- **Definition:**
  Specification involves documenting the requirements in a clear, structured, and unambiguous format that serves as a reference for all project stakeholders.
- **Examples of Documentation Formats:**
  1. **Use Case Diagrams:**
     Visual representations of interactions between users and the system to achieve specific goals.
  2. **User Stories:**
     Short, simple descriptions of a feature from the perspective of the end user.
  3. **Software Requirements Specification (SRS):**
     A detailed document that outlines functional and non-functional requirements, system constraints, and other relevant details.

Validation

- **Definition:**
  Validation ensures that the documented requirements meet user needs and expectations and are complete, consistent, and feasible.
- **Techniques:**
  1. **Reviews and Inspections:**
     Conducting walkthroughs and reviews with stakeholders and team members to verify requirements.
  2. **Prototyping:**
     Creating a preliminary version of the system to demonstrate functionality and gather feedback.
  3. **Test Cases:**
     Writing test cases based on requirements to verify their completeness and correctness during development.

Management

- **Definition:**
  Management involves handling changes to requirements throughout the project lifecycle, ensuring that updates align with stakeholder needs and do not disrupt the project.
- **Tools and Techniques:**
  1. **Version Control Systems:**
     Tools like Git to track changes to requirement documents and maintain version history.
  2. **Requirements Management Software:**
     Tools like Jira or IBM Rational DOORS to organize, prioritize, and track requirements changes.
  3. **Change Control Process:**
     A formal process for evaluating, approving, and documenting requirement changes to ensure traceability.

Introduction to Types of Requirements

- **Overview:**
  Requirements are the foundation of software development, guiding the design, implementation, and validation of a system. They can be broadly classified into two main categories:

  1. **Functional Requirements:**
     Define the specific functionalities or features the system must provide.
  2. **Non-Functional Requirements:**
     Define the quality attributes or performance criteria the system must meet.

Functional Requirements

- Functional requirements specify what the system should do to meet user needs and achieve business goals. They describe the behavior of the system in response to inputs and interactions.

- **Key Characteristics:**
  - Directly related to user interactions.
  - Clearly define inputs, processes, and outputs.
  - Serve as the basis for system design and testing.

Functional Requirements

- **Examples:**
  1. **Login Functionality:**
     The system should allow users to log in with a username and password.
  2. **Search Feature:**
     The system should provide a search bar that returns relevant results based on user queries.
  3. **Report Generation:**
     The system should generate monthly sales reports in PDF format.

Non-Functional Requirements

- Non-functional requirements describe how the system should perform rather than what it should do. These requirements focus on quality attributes, ensuring the system operates efficiently and meets user expectations.

- **Key Characteristics:**
  - Address operational and environmental aspects.
  - Impact user satisfaction and system reliability.
  - Often harder to measure and validate.

Non-Functional Requirements

- **Examples:**
  1. **Performance:**
     The system should handle 1,000 concurrent users without performance degradation.
  2. **Security:**
     The system should encrypt user data using AES-256 encryption.
  3. **Scalability:**
     The system should support increasing workloads by adding additional servers.
  4. **Usability:**
     The system should have an intuitive user interface with accessible navigation.

Comparison: Functional vs. Non-Functional Requirements

Aspect	Functional Requirements	Non-Functional Requirements
Definition	What the system should do	How the system should perform
Focus	Specific functionality and tasks	Quality attributes and performance
Examples	Login, search, report generation	Performance, security, scalability, usability
Measurement	Measured through system functionality	Measured through system benchmarks
Impact	Directly impacts system operations	Impacts user satisfaction and system efficiency

Importance of Clear Requirements

- **Functional Requirements:**
  - Clearly defined functional requirements ensure the system fulfills user tasks effectively.
  - They reduce ambiguity and guide the design and implementation processes.
  - Example: If the login functionality is unclear, users may experience frustration or errors.
- **Non-Functional Requirements:**
  - These requirements ensure the system performs well under real-world conditions.
  - They directly impact user satisfaction, reliability, and scalability.
  - Example: A slow or unresponsive system can lead to poor user experiences, even if all functional requirements are met.

What Makes a Good Requirement?

- **Characteristics of a Good Requirement:**
  1. **Clear:**
     The requirement should be easy to understand and leave no room for misinterpretation.
     1. Example: "The system shall generate a weekly sales report in PDF format."
  2. **Concise:**
     The requirement should be brief and to the point while still providing enough detail.
  3. **Testable:**
     The requirement must be verifiable through testing or demonstration.
     1. Example: "The system shall encrypt all user passwords using AES-256 encryption."
  4. **Consistent:**
     There should be no contradictions between requirements or other project documents.
  5. **Feasible:**
     The requirement should be achievable within the project's constraints of time, budget, and technology.

Common Pitfalls in Writing Requirements

- **Ambiguity:**
  Requirements that can be interpreted in multiple ways lead to confusion.
- Example of a poor requirement: "The system should be fast."
- **Vagueness:**
  Requirements lacking specific details are difficult to implement and test.
- Example: "The system should handle many users."
- **Lack of Detail:**
  Insufficient information leaves developers guessing about what is needed.
- **Over-Specification:**
  Overly detailed requirements can limit flexibility and creativity in implementation.
- Example: Specifying exact implementation methods instead of expected outcomes.

Using Templates and Standards

- **Ensures Uniformity:**
  All requirements are documented in a consistent format, making them easier to understand and review.
- **Promotes Completeness:**
  Templates guide writers to include all necessary information, reducing the risk of missed details.
- **Enhances Quality:**
  Standards ensure that requirements meet industry best practices.

Examples of Well-Written Requirements

- **Functional Requirement:**
  - **Good Example:** "The system shall allow users to reset their password using their registered email address."
  - **Why it's good:** It's clear, specific, and testable.
- **Non-Functional Requirement:**
  - **Good Example:** "The system shall support up to 100 concurrent users with a response time of less than 2 seconds for any operation."
  - **Why it's good:** It defines measurable performance criteria.

Techniques to Improve Requirement Quality

- **Use Simple Language:**
  Write requirements in plain, non-technical language that all stakeholders can understand.
- Example: Instead of "implement an asymmetric cryptographic algorithm," write "use public-private key encryption."
- **Avoid Technical Jargon:**
  Use terminology that is accessible to both technical and non-technical stakeholders.
- **Engage Stakeholders in Reviews:**
  Regularly review requirements with stakeholders to ensure alignment and avoid miscommunication.
- **Utilize Visual Aids:**
  Incorporate diagrams, use case models, or flowcharts to supplement textual requirements.
- **Iterative Refinement:**
  Continuously refine requirements based on stakeholder feedback and evolving project needs.

"Great software isn’t magic—it’s built on clear goals, solid plans, and knowing what matters most. Nail the basics, and the rest will flow."

End of Topic 1

EC3290 – Software Requirements Engineering

Topic 1 – Understanding, Eliciting, and Documenting Software Requirements

CLO 1

4 Hours

Requirements Inception

- The initial phase of gathering high-level requirements and understanding the problem domain.
- Understand business needs and goals
- Identify key stakeholders
- Define high-level system scope
- Outcome is a clear understanding of the problem to be solved.

Domain Understanding

- Learning about the industry, business processes, and challenges.
- Understanding domain-specific terminology.

- How to Achieve It?
  - Studying existing systems and documentation.
  - Interviewing domain experts.
  - Observing real-world business operations.

Product Vision

- A high-level statement that describes what the product will achieve.
- Key Elements of Product Vision:
  - Target Users
  - Business Value
  - Competitive Advantage

"An AI-powered chatbot to automate customer service and reduce response time by 50%."

Defining the Scope

- Defines what is included (and excluded) in the system.
- Helps avoid **scope creep**.
- Scope Statement Components:
  - System Features
  - Boundaries & Constraints
  - Assumptions

"The system will allow users to register, log in, and make purchases, but will not handle inventory management."

Requirements Elicitation

- The process of gathering requirements from stakeholders.

- **Challenges in Elicitation:**
  - Unclear business needs
  - Conflicting stakeholder interests
  - Difficulty in expressing requirements

Interview Technique

- An interview is a conversational method where stakeholders provide insights into their needs and expectations for the system.
- Types of Interviews:
  - Structured: Predefined questions for consistency.
  - Semi-Structured: Combination of standard questions with some flexibility.
  - Unstructured: Open-ended conversation for in-depth exploration.

Structured Interview

- In a structured interview, the interviewer follows a set of predefined questions. Every participant answers the same questions, making the process standardized.
- This format ensures consistency and makes it easier to compare responses. However, it may limit in-depth exploration since the questions are fixed.

Semi-Structured Interview:

- Semi-structured interviews use a combination of standard questions and allow some flexibility.
- The interviewer follows a basic framework of questions but can probe further or ask follow-up questions based on the respondent’s answers.
- This approach balances consistency with flexibility, allowing deeper insights while maintaining some structure.

Unstructured Interview:

- In unstructured interviews, the conversation is open-ended, with no strict questions or order. The interviewer may have a few topics in mind but lets the conversation flow naturally.
- This approach provides the most flexibility and is ideal for gaining detailed insights and exploring topics in depth.
- However, it’s harder to compare responses across participants since the conversation varies widely from one person to another.

Benefits of each:

- **Structured** is best for consistency and comparison.
- **Semi-structured** works well when a balance of structure and flexibility is needed.
- **Unstructured** is ideal for in-depth exploration but requires skilled interviewers to keep the conversation productive.

Preparing for an Interview

DEFINE INTERVIEW OBJECTIVES

IDENTIFY INTERVIEWEES

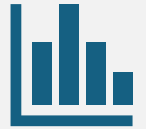
PREPARE QUESTIONS TO COVER KEY AREAS OF INQUIRY.

Conducting an Interview

BEGIN WITH AN INTRODUCTION

ASK QUESTIONS

ENCOURAGE DISCUSSION



RECORD RESPONSES FOR LATER ANALYSIS.

Benefits and Limitations

Benefits:

Gathers in-depth information, clarifies expectations.

Limitations:

Time-consuming, may introduce interviewer bias.

Questionnaire Technique

- A questionnaire is a structured form that stakeholders fill out to provide input on their needs and requirements.
- Types of Questionnaires:
  - Open-ended: Allows respondents to provide detailed feedback.
  - Close-ended: Limits answers to specific choices for easier analysis.
- Designing Effective Questionnaires:
  - Use clear, concise questions, include a mix of question types, and avoid leading questions.
- Distributing and Collecting Responses:
  - Send via email, post on an online survey tool, or distribute in person to gather responses.

Examples

Open Ended Question	Close Ended Question
"What challenges do you face when using this system, and how do you think it could be improved?"	"How satisfied are you with the system's ease of use?" • Very Satisfied • Satisfied • Neutral • Dissatisfied • Very Dissatisfied

Benefits and Limitations

Benefits:

Efficient for large groups, anonymous feedback.

Limitations:

Limited depth, may lack clarity.

Observation Technique

- Observation involves watching users interact with the system or perform tasks to gain insights into their behaviors and challenges.
- Types of Observation:
  - Direct: Observer is present but does not interact.
  - Participant: Observer interacts with the subjects and environment.
- Steps for Effective Observation:
  - Plan objectives, define scope, and record observations systematically.

Direct Observation

- In direct observation, the observer watches the users but does not engage or interact with them.
- The goal is to see how users naturally interact with the system or perform tasks without influencing their behavior.
- This approach is helpful for identifying typical user behavior and any issues that might not be obvious in interviews or questionnaires.

Participants Observations

- In participant observation, the observer actively engages in the environment, working alongside the users and sometimes even performing the same tasks.
- This method allows the observer to gain a deeper, firsthand understanding of user challenges, needs, and the context in which tasks are performed.
- It’s useful for uncovering insights that might only come from experiencing the work environment directly.

Benefits and Limitations

Benefits:

Captures real user behavior, identifies hidden needs.

Limitations:

Observer bias, limited to observable actions.

Document Reading Technique

- Document reading involves reviewing existing documents (manuals, reports, and records) to understand system requirements.
- **Types of Documents Used**:
  - Manuals, technical specifications, historical records, policy documents.
- **Process of Document Analysis**:
  - Identify relevant documents, review contents, and note key information for requirements.

Benefits and Limitations

- Benefits:
  - Provides historical context, no stakeholder availability needed.

- Limitations:
  - May be outdated, lacks interactive clarification.

Requirements Documentation

- Acts as a reference for development and testing.
- Prevents miscommunication.
- Types of Requirement Documents:
  - Software Requirements Specification (SRS)
  - Use Case Descriptions
  - User Stories

Requirements Documentation

- Characteristics of Good Requirements:
  - Clear
  - Complete
  - Consistent
  - Feasible

"The system shall allow users to reset their passwords via email verification."

End of Chapter 2

EC3290 – Software Requirements Engineering

Topic 3 – Requirements Analysis, Modeling, and Specification

Learning Objectives

- Understand the difference between **description and specification**.
- Learn various **modeling techniques and diagrammatic notations**.
- Explore **Use Case Modeling and Scenario Descriptions**.
- Gain insights into **domain analysis and requirements understanding**.
- Introduce **formal specification techniques**.

Introduction to Requirements Analysis

- Process of determining user expectations for a new or modified system.
- Prevents requirement misunderstandings.
- Ensures better system design.
- Reduces project failure risk.

Description vs. Specification

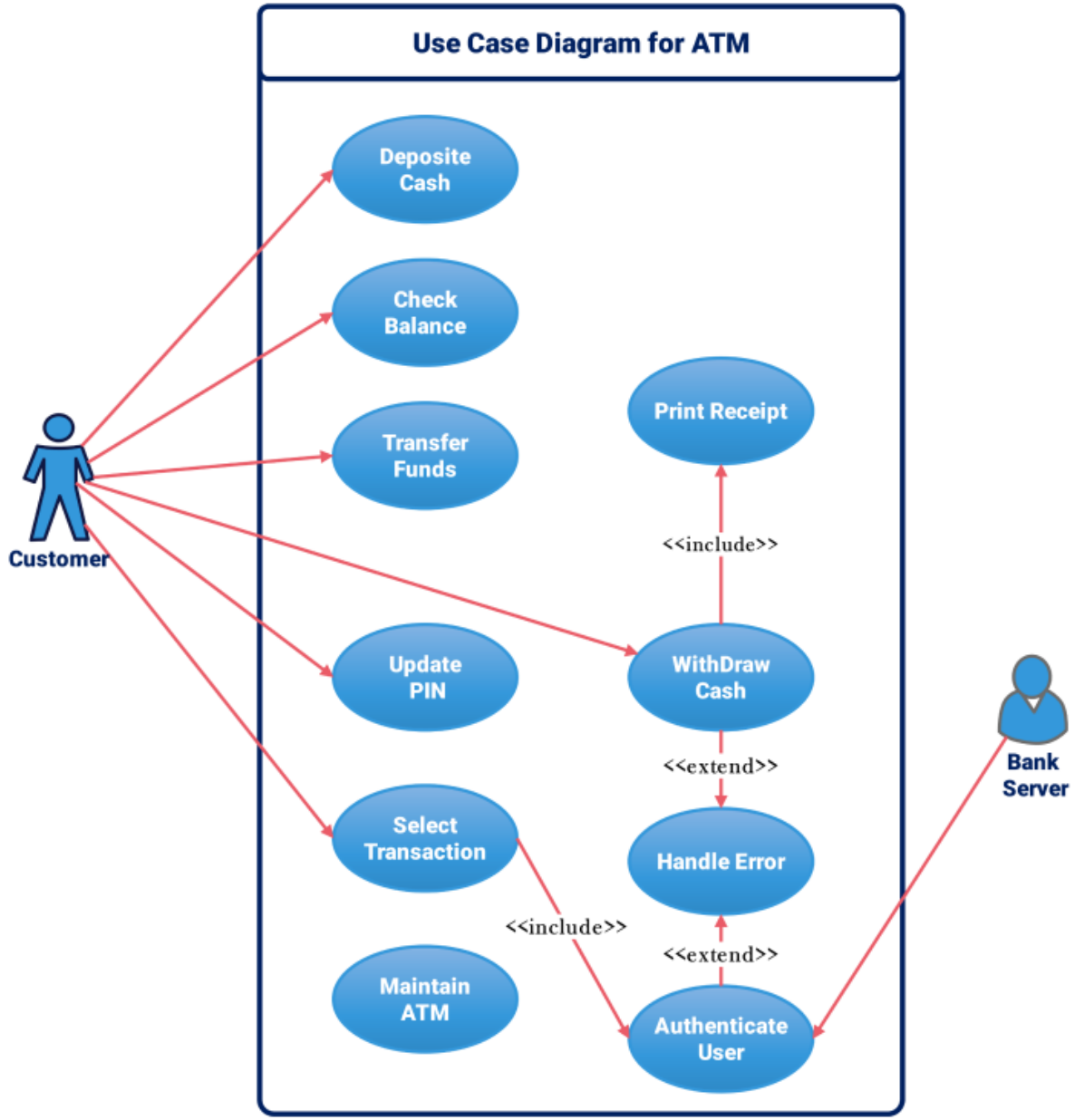
- **Description:**
  - Natural language explanation of system behavior.
  - Often informal and ambiguous.
- **Specification:**
  - Precise and structured representation of system behavior.
  - Uses models, diagrams, and formal notations.
- **Example:**
  - Description: "The system should allow users to login."
  - Specification: "The system shall authenticate users using a valid username and password stored in the database."

Requirements Modelling Techniques

- **Purpose of Modelling:**
  - Visual representation of complex system requirements.
  - Enhances understanding and communication.
- **Key Techniques:**
  - **Use Case Modelling**
  - **Data Flow Diagrams (DFD)**
  - **Entity-Relationship Diagrams (ERD)**
  - **Class Diagrams (UML)**

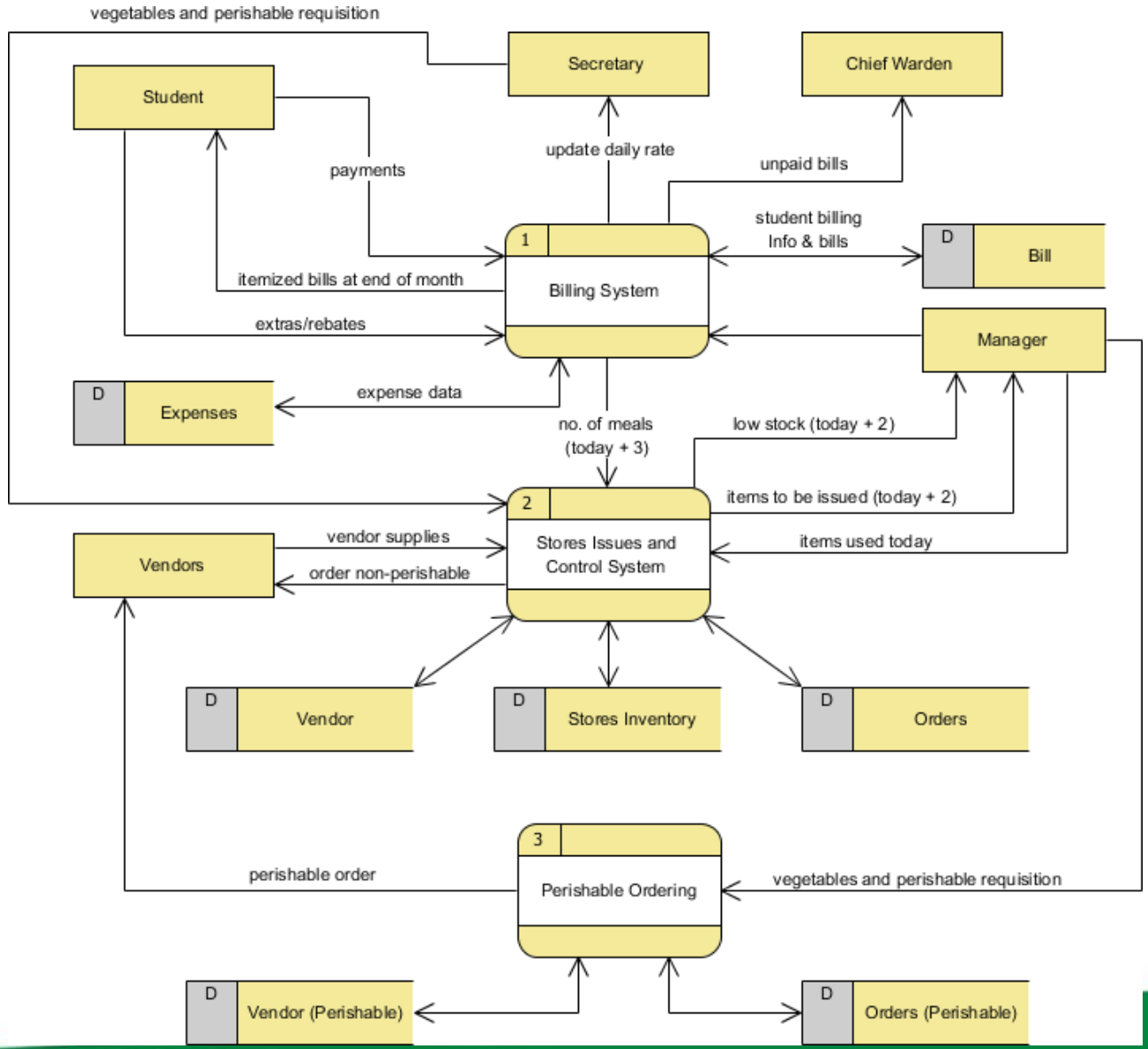
Use Case Modelling and Scenario Descriptions

- **Use Case Diagram:**
  - Captures interactions between users and the system.
  - Identifies system functionality.
- **Scenario Descriptions:**
  - Details different paths in system operation (Normal, Alternative, Exception flows).


Use Case Diagram for ATM
Deposite Cash
Check Balance
Transfer Funds
Print Receipt
<<include>>
Customer
Update PIN
WithDraw Cash
<<extend>>
Bank Server
Select Transaction
Handle Error
<<include>>
<<extend>>
Maintain ATM
Authenticate User

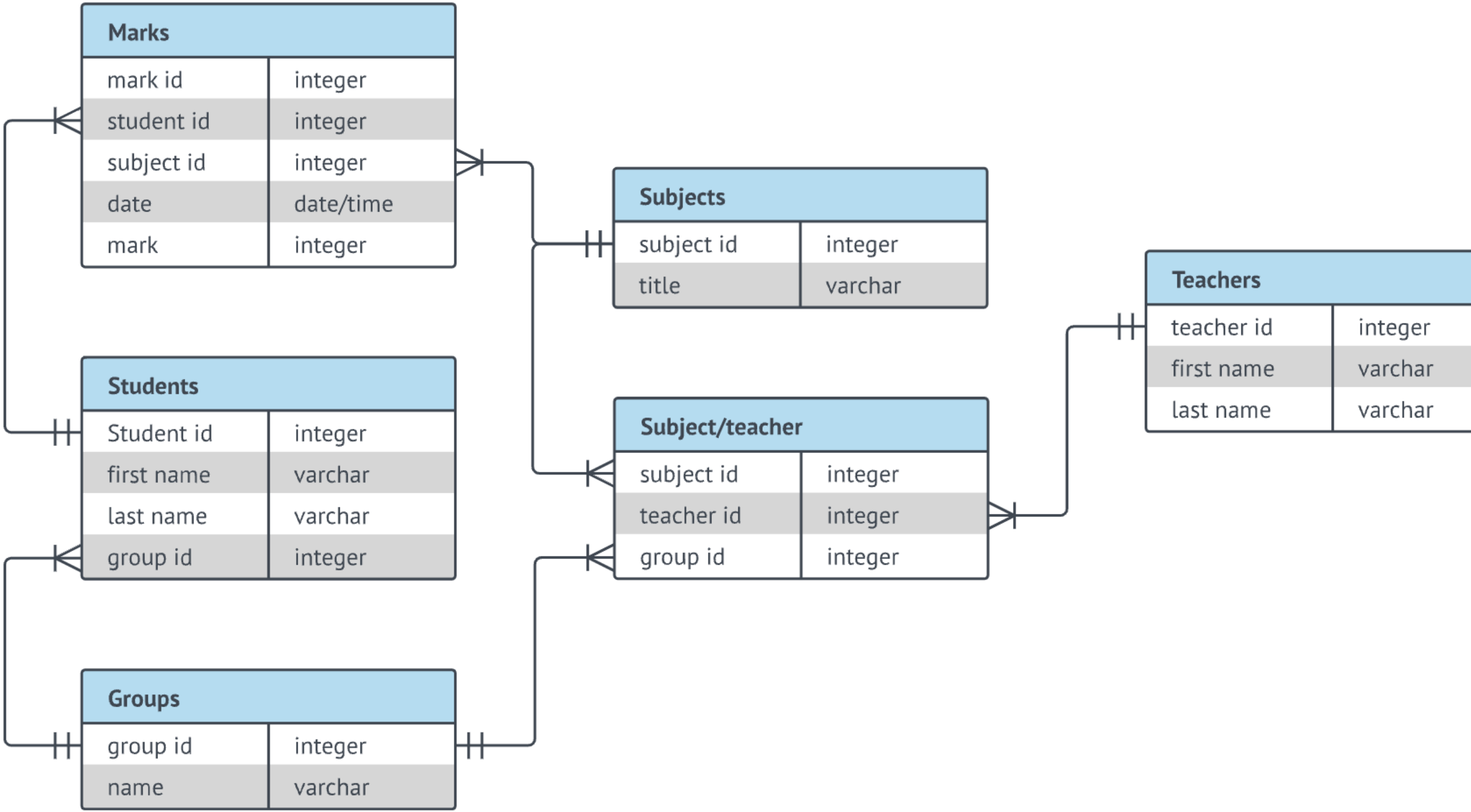
Data Flow Diagrams (DFD)

- Represents how data moves through a system.
- **Components:**
  - **Processes** (Circle)
  - **Data Stores** (Rectangle)
  - **External Entities** (Square)
  - **Data Flows** (Arrows)


vegetables and perishable requisition
Secretary
Chief Warden
Student
update daily rate
payments
unpaid bills
1
Billing System
student billing
Info & bills
D
Bill
itemized bills at end of month
extras/rebates
Manager
D
Expenses
expense data
no. of meals
(today + 3)
low stock (today + 2)
2
Stores Issues and
Control System
items to be issued (today + 2)
items used today
Vendors
vendor supplies
order non-perishable
D
Vendor
D
Stores Inventory
D
Orders
3
Perishable Ordering
perishable order
vegetables and perishable requisition
D
Vendor (Perishable)
D
Orders (Perishable)

Entity-Relationship Diagram (ERD)

- **Definition:** Diagram that models database structure.
- **Key Elements:**
  - Entities (Objects like Students, Courses)
  - Attributes (Properties like Student Name, Course Code)
  - Relationships (Associations like Enrolled-In)


Marks
mark id
integer
student id
integer
subject id
integer
date
date/time
mark
integer
Subjects
subject id
integer
title
varchar
Teachers
teacher id
integer
first name
varchar
last name
varchar
Students
Student id
integer
first name
varchar
last name
varchar
group id
integer
Subject/teacher
subject id
integer
teacher id
integer
group id
integer
Groups
group id
integer
name
varchar

Analysis for Understanding the Domain and Requirements

- **Domain Analysis:**
  - Identifying system environment, stakeholders, and constraints.
  - Techniques: **Interviews, Questionnaires, Document Analysis**.
- **Requirement Analysis Approaches:**
  - Structured Analysis (DFD, ERD)
  - Object-Oriented Analysis (UML Diagrams, Class Diagrams)

Use Case Description Format

- **Use Case Name:** Login Process
- **Actors:** User, Authentication System
- **Preconditions:** User must have an account
- **Main Flow:**
  - User enters username and password.
  - System verifies credentials.
  - User is granted access.
- **Alternative Flow:** Wrong password → Show error message.

Formal Specification

- **Definition:** Mathematically-based techniques to specify software requirements.
- **Why is it useful?**
  - Removes ambiguity.
  - Improves verification and validation.
- **Common Methods:**
  - **Z-Notation, B-Method, VDM (Vienna Development Method)**

Benefits of Using Formal Specification

- Precise and unambiguous.
- Reduces misinterpretation.
- Helps with system validation before development.

Challenges in Requirements Analysis

- **Common Issues:**
  - Unclear or vague requirements.
  - Changing requirements.
  - Conflicting stakeholder interests.
- **How to overcome them?**
  - Regular communication with stakeholders.
  - Using prototypes and visual models.

Real-World Example – Software Requirement Failure

- **Case Study:** London Ambulance System Failure
  - System designed without clear requirements.
  - Lack of proper analysis led to incorrect dispatching.
  - Lesson: Proper requirement engineering prevents system failures.

Industry Best Practices in Requirements Engineering

- **Define SMART Requirements:**
  - Specific, Measurable, Achievable, Relevant, Time-bound.
- **Use Requirement Management Tools:**
  - JIRA, IBM Rational DOORS, ReqSuite.

Summary

- **Requirement Analysis** ensures clear, complete, and correct requirements.
- **Modeling Techniques** help visualize requirements for better understanding.
- **Use Case Modeling** captures user interactions.
- **Formal Specification** provides a rigorous, mathematical approach to defining requirements.

End of Topic 3

EC3290 – Software Requirements Engineering

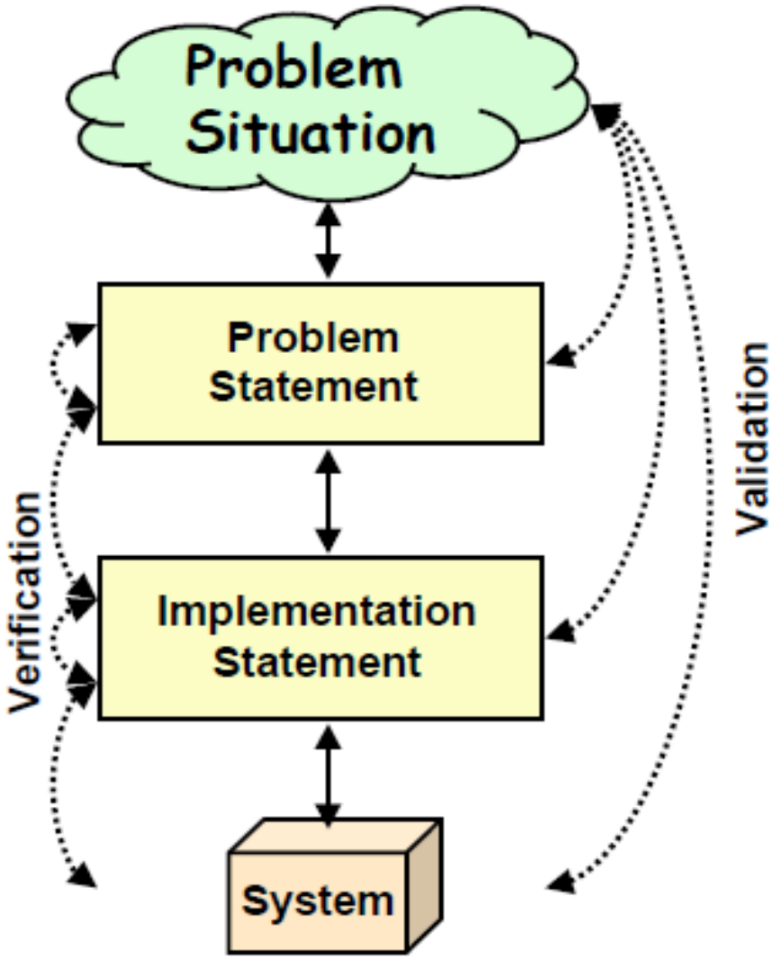
Topic 4 – Software Verification and Validation

Learning Objectives

- Understand the difference between verification and validation
- Explore techniques used in requirement V&V
- Learn how to identify conflicts, inconsistencies, and incompleteness in requirements

What is Requirements V&V?

- **Verification**
  - Check if the requirement document is correct and logically structured.
  - Are we building the product right?
- **Validation**
  - Confirm if the documented requirements truly reflect the needs of stakeholders.
  - Are we building the right product?



Importance of V&V

- Reduces costly rework in later stages
- Ensures customer satisfaction
- Improves software quality

- **Example:** NASA lost a $125 million spacecraft because of a requirements error. That's one expensive typo.

Example 1

YAHOO! MAIL

- Incident: Yahoo! mail doesn't let me log in
- Failure: The user account cannot be accessed in the user database.
- Fault: The user database can not be reached.
- Error: There was no backup user database in the system.

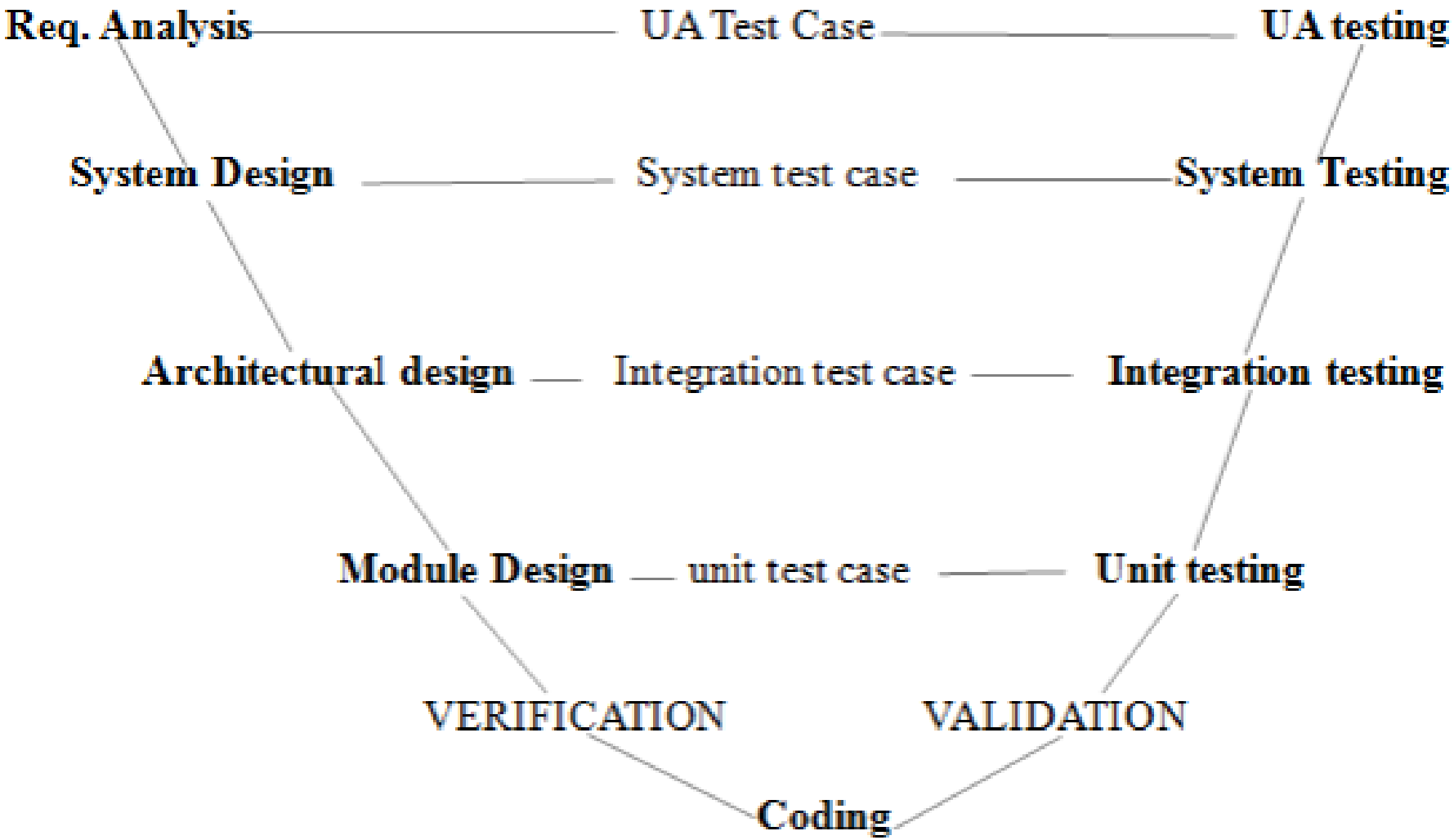
Example 2

- Fatal Therac-25 X-ray Radiation
- In 1986, a man in Texas received between 16,500-25,000 radiations in less than 10 sec, over an area of about 1 cm.
- He passed away 5 months later.
- The root cause of the incident was a SW failure ☹

- Incident: A patient passed away
- Failure: The device applied higher frequency of radiations than what was safe. Safety range: [1…10,000 Hz].
- Fault: The software controller of the device did not have a conditional block (if …. else statements) to perform range checking on the frequency of the radiation to be applied.
- (2) Errors:
  1. The SW developer of the device controller system had forgotten to include a range checking conditional block on the frequency of the radiation to be applied.
  2. The device operator was NOT supposed to enter anything outside [1…10,000 Hz] range.

When Does V&V Happen?

- Throughout the software development life cycle (SDLC)
- Especially during and after requirements elicitation and specification
- Before design and implementation begin


Req. Analysis
UA Test Case
UA testing
System Design
System test case
System Testing
Architectural design
Integration test case
Integration testing
Module Design
unit test case
Unit testing
VERIFICATION
VALIDATION
Coding

Verification: In-Depth

- Ensures the software meets the **specified** requirements
- Focuses on correctness and consistency
- Techniques: Reviews, Walkthroughs, Inspections

Validation: In-Depth

- Ensures the software fulfills **user needs and expectations**
- Involves stakeholders
- Techniques: Prototyping, User Feedback, Acceptance Testing

The V&V Relationship

- **Verification = Internal check**
- **Validation = External check**
- Both are essential; one without the other can lead to disaster

Verification

- **Definition**
  Verification refers to the set of activities that ensure software correctly implements the specific function.
- **Focus**
  It includes checking documents, designs, codes, and programs.

VS

Validation

- **Definition**
  Validation refers to the set of activities that ensure that the software that has been built is traceable to customer requirements.
- **Focus**
  It includes testing and validating the actual product.

V&V Activities in Requirements Phase

- Requirements review
- Consistency checks
- Prototyping for validation
- Traceability analysis

Inspection Techniques

- Formal, peer-based examination of documents
- Checklist-driven
- Focus on:
  - Ambiguities
  - Conflicts
  - Incompleteness

Walkthroughs

- Informal group discussion
- Presenter leads; others provide feedback
- Goal: Understand and evaluate the requirements

Reviews

- Technical review meetings
- Used to detect errors early
- May be formal or informal

- **Example:** Weekly review sessions with client or internal QA team.

Prototyping for Validation

- Build a mock-up of the system or interface
- Gather feedback from users
- Helps discover hidden needs

- Because "I'll know it when I see it" is not a requirement.

Acceptance Criteria and Testing

- Define what it means for a requirement to be "done"
- Involve end-users in writing test cases
- Must be measurable and testable

Conflict Detection in Requirements

- **Conflict:** Two or more requirements contradict each other
- **Example:** One requirement says “System must run in 2 seconds,” another says “Must perform deep analysis.”
- **Solution:** Use traceability and conflict matrices.

Inconsistency Detection

- Different parts of the requirement contradict or don’t align
- **Example:** “All users must log in” vs. “Guest users can access without login”
- **Technique:** Cross-checking related requirements.

Completeness Check

- Ensures all required functionalities are described
- No missing constraints or conditions

- **Checklist:** Are all stakeholders' needs addressed? Are all interfaces defined?

Ambiguity Detection

- Ambiguous: “The system should be fast”
- Clear: “System response time shall not exceed 2 seconds under load”

Traceability Matrix

Maps requirements to:

- Stakeholder needs
- Design components
- Test cases
  Benefit: Helps detect missing or extra functionality
  Example table: Req1 → Use Case 1 → Module A → Test Case 5

Requirements Traceability Matrix

Project Name	<Project Name here>	Created On	3-Oct-11	Reviewed On	4-Oct-11
Release No	<Project Release>	Created By	<Creater's Name>	Reviewed By	<Reviewer's Name>
Version	<Doc version>				

ID	Requirement ID	Requirement Description	Status	Design Document	Code Module	TestCase ID	Test Case Name	User Manual	Tested On/ Verification
001	UC 1.0	Testing Requirement Description here. It should not be more than 2-3 lines	Status	DM-001	CM-001	TC-001	ProjName_UCID_TestCase Name	Section 4.5	Pending
002	UC 1.1	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-002	CM-002	TC-002 TC-003	N.A	Section 4.6	Verified
003	UC 1.2	Testing Requirement Description here. It should not be more than 2-3 lines	Status	DM-003	CM-003	TC-004 TC-006 TC-007 TC-008		Section 5.7	Verified
004	UC 1.3	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-004	CM-004			Section 6.8	In-progress
005	UC 1.4	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-005	CM-005			Section 7.9	Not Verified
006	UC 1.5	Testing Requirement Description here. It should not be more than 2-3 lines	TBD	DM-006	CM-006			Section 4.10	Verified
007	UC 1.6	Testing Requirement Description here. It should not be more than 2-3 lines	Approved					Section 4.11	Not Verified
008	UC 1.7	Testing Requirement Description here. It should not be more than 2-3 lines	TBD					Section 4.12	Pending
009	UC 1.8	Testing Requirement Description here. It should not be more than 2-3 lines	TBD					Section 4.13	Not Verified
010	UC 1.9	Testing Requirement Description here. It should not be more than 2-3 lines	Approved					Section 4.14	Pending

Tool Support for V&V

Tools like:

- IBM DOORS
- ReqView
- JIRA (for issue tracking)

- Help automate conflict detection and versioning

V&V in Agile Context

- Continuous validation via sprints and reviews
- Definition of Done (DoD) ensures verification
- User stories and acceptance criteria help validate

Common Mistakes in V&V

- Skipping reviews
- Relying too much on tools
- Not involving users in validation

Summary

- V&V ensure correct and complete requirements
- Verification = “Did we build it right?”
- Validation = “Did we build the right thing?”
- Use a variety of techniques: inspection, prototyping, testing

EC3290 – Software Requirements Engineering

Topic 5 – Requirement Evolution and Management

Learning Objectives

This session provides a complete overview of how to manage evolving requirements in software development. You will understand the lifecycle of a requirement and how to manage it effectively as situations change.

Key Points:

- Understand the evolving nature of software requirements
- Learn to apply requirements traceability techniques
- Practice prioritization methods like MoSCoW and Kano
- Explore how to manage changes using baselines
- Understand the value of reusing requirements

Introduction to Requirements Evolution

Software requirements are not static. They must adapt to changing environments, new stakeholder needs, and unforeseen challenges. Requirements evolution is a natural part of the software lifecycle.

- Requirements change due to business and technical drivers
- The need for flexibility in planning and documentation
- Helps avoid outdated or irrelevant product features

What Causes Requirements to Evolve?

Several internal and external factors influence the need to update software requirements. These changes can have significant impacts on project scope and delivery.

Key Causes:

- Stakeholder feedback
- Regulatory updates
- Competitive market demands
- Technical limitations

Real-World Example - Mobile Banking App

A mobile banking app initially supports only local transactions. Later, the client requests international transfer functionality due to business expansion. Requirements evolve to include exchange rates, compliance checks, and international APIs.

Key Points:

- Reflects how growth changes user needs
- Shows how even small additions can trigger complex changes

Requirements Traceability

Traceability helps track each requirement from origin to deployment. It ensures nothing is missed and every stakeholder need is fulfilled.

Key Points:

- Supports impact analysis and auditing
- Links requirements to design, code, and tests
- Improves accountability and verification

Types of Traceability

- Forward Traceability: Requirement → Test
- Backward Traceability: Test → Requirement
- Bidirectional Traceability: Both directions

Traceability Example - Password Reset

A requirement like "User resets password via OTP" can be traced through design documents, code modules, and test cases.

Trace Links:

- Design: OTP input interface
- Code: otp_reset.py
- Test: TC003
- Deploy: Version 1.2

Tools for Traceability

- IBM DOORS
- Jira with Xray/Zephyr
- Jama Connect
- ReQtest

Requirements Prioritization

Not all requirements are created equal. Prioritization helps teams focus on delivering the highest value first.

Benefits:

- Aligns with business goals
- Manages scope effectively
- Enhances decision-making in trade-offs

MoSCoW Technique

MoSCoW is a simple but effective prioritization method.

Categories:

- Must have: Essential
- Should have: Important
- Could have: Nice to have
- Won’t have: Out of scope (for now)

Kano Model and 100-Dollar Test

Kano Categories:

- Basic Needs
- Performance Features
- Delighters

100-Dollar Test:

- Allocate imaginary budget to requirements
- Reflects perceived stakeholder value

Value vs. Complexity Matrix

Helps visualize the trade-off between value and implementation difficulty.

Quadrants:

- High value, low effort: Prioritize
- High value, high effort: Plan with caution
- Low value, low effort: Consider
- Low value, high effort: Avoid

Example - Mobile Banking App Features

Feature	Priority
Login/Signup	Must
View Balance	Must
Fund Transfer	Should
Budget Planner	Could
Crypto Trading	Won’t

Why Requirements Change

Change is inevitable in software development. Recognizing and managing change is critical.

Reasons:

- New insights or feedback
- Legal compliance
- Feature gaps
- New technologies

Change Management Process

Steps:

1. Submit change request
2. Analyze impact
3. Review with stakeholders
4. Approve or reject
5. Update baseline and notify team

Requirements Baselines

A baseline freezes the requirements at a given time. All future changes must follow formal procedures.

Benefits:

- Provides a reference point
- Supports project tracking
- Controls scope creep

Configuration Management

Includes:

- Version control (e.g., Git)
- Change logs
- Release notes

Benefits:

- Reduces confusion
- Tracks evolution over time

Reusing Requirements

- Reusability leads to efficiency, especially for common modules.

Examples of Reusable Requirements:

- Login authentication
- Email notifications
- Role-based access

Advantages:

- Saves time
- Improves reliability

Recap

- Requirements are living documents. Managing them means tracking, prioritizing, controlling, and reusing.
- Traceability ensures visibility
- Prioritization helps decisions
- Change must be controlled
- Baselines keep projects stable
- Reuse builds on past work

EC3290 – Software Requirements Engineering

Topic 5 – Advanced Topics in Software Requirement Engineering

Topics:

- Hazard Analysis & Safety-Critical Requirements
- Threat Modeling & Security-Critical Requirements
- Tool Support in Requirements Engineering
- Open Research Challenges

Introduction to Advanced Topics

- As modern software systems become increasingly embedded in critical environments—from healthcare and transportation to finance and defense—the field of requirements engineering must evolve. Four advanced areas that reflect the growing demand for dependable, secure, and maintainable systems.

Hazard Analysis and Safety-Critical System Requirements

Safety-critical systems are those where failures can result in catastrophic consequences: human injury, death, or major financial loss. Think of pacemakers, aircraft autopilot systems, or nuclear power controls.

The goal of requirements engineering in such systems is not just to ensure functionality—but to safeguard against harmful scenarios.

Hazard Analysis?

Hazard analysis is the structured process of identifying conditions or events that could lead to system failure or harm.
It helps us answer:

- What can go wrong?
- What are the consequences?
- What measures can prevent it?

This early-stage analysis directly influences how safety requirements are written.

Techniques in Hazard Analysis

Three popular techniques are widely used in industry:

- **FMEA** (Failure Modes and Effects Analysis): Assesses how different parts might fail and the impact.
- **FTA** (Fault Tree Analysis): A top-down diagram-based method to analyze cause-and-effect.
- **HAZOP** (Hazard and Operability): A systematic team review to find deviations from intended operations.

Failure Modes and Effects Analysis (FMEA)

FMEA is a **bottom-up** approach that identifies potential failure modes of each component, analyzes their consequences, and prioritizes them based on severity, occurrence, and detectability.

Steps in FMEA:

1. List components and functions
2. Identify how each can fail (failure modes)
3. Analyze effects of each failure
4. Assign Risk Priority Number (RPN)
5. Recommend corrective actions

Failure Modes and Effects Analysis (FMEA)

Example:

In a **heart monitor** system:

- Component: ECG sensor
- Failure Mode: Sensor disconnect
- Effect: No reading → risk to patient
- RPN: High → Add alert system when no signal detected

Fault Tree Analysis (FTA)

FTA is a **top-down** graphical technique that starts from an undesired event and uses logic gates (AND, OR) to trace the possible root causes.

Steps in FTA:

1. Define the undesired "top event"
2. Break down causes using logic gates
3. Identify minimal cut sets (combinations that can trigger the top event)
4. Analyze probabilities and dependencies

Fault Tree Analysis (FTA)

Example:

Top event: “Aircraft loses power”

- Cause A: Fuel exhaustion
- Cause B: Engine control software crash
  FTA builds a visual tree showing how these causes interact.

Hazard and Operability Study (HAZOP)

HAZOP is a **structured team-based** technique that analyzes potential deviations from the intended design using guide words like "more," "less," "none," etc.

Steps in HAZOP:

1. Form a multidisciplinary team
2. Divide system into nodes/processes
3. Apply guide words to each node (e.g., "No flow", "More pressure")
4. Brainstorm possible causes and consequences
5. Document actions needed

Hazard and Operability Study (HAZOP)

In a **chemical plant**:

- Node: Pipeline transporting gas
- Guide word: “More pressure”
- Deviation: Pressure above safety threshold
- Consequence: Explosion risk
- Action: Add pressure relief valve

Comparison of Hazard Analysis Techniques

Technique	Approach	Focus	Best For	Strengths	Limitations
FMEA	Bottom-up	Component-level failures	Mechanical/electrical systems	Simple to apply, quantifies risk	May miss system-level interactions
FTA	Top-down	Undesired events & causes	Aerospace, nuclear, automotive	Clear visualization of failure logic	Complex for large systems
HAZOP	Team-based with guide words	Process deviations	Chemical/process industries	Encourages brainstorming, very thorough	Time-consuming, needs expertise

From Hazards to Requirements

Once hazards are known, we specify how the system should respond or prevent these situations.
For example:

- Hazard: Sensor overheating
- Requirement: “The system shall shut down the module if temperature exceeds 80°C.”

These are **non-functional requirements**, often focusing on system behavior under stress or failure.

Threat Modeling and Security-Critical Requirements

- Why Security Matters
  - Security is now an essential requirement in almost every system. From personal apps to critical infrastructure, breaches can expose private data or even endanger lives.

  - Unlike hazards (accidental), threats are **intentional**, posed by attackers. Our role is to anticipate and mitigate them through proper requirements.

Threat Modeling

Threat modeling is a proactive approach to uncovering and analyzing security threats during the design phase.

- Steps include:

1. Identifying assets (data, user credentials, etc.)
2. Listing potential attackers and their motives
3. Pinpointing vulnerabilities
4. Defining countermeasures

STRIDE Framework

The STRIDE model categorizes six types of threats:

- **Spoofing**: Impersonating identities
- **Tampering**: Modifying data
- **Repudiation**: Denying actions
- **Information Disclosure**: Leaking data
- **Denial of Service**: Disrupting services
- **Elevation of Privilege**: Gaining unauthorized access

For each threat type, we derive one or more security requirements.

S – Spoofing Identity

What is Spoofing?
When someone pretends to be someone else to gain unauthorized access.

Example:

- A hacker logs into a system using someone else's username and password.

Analogy:

Like someone using a fake ID to enter a restricted event pretending to be someone else.

Real-world Scenario:

- An attacker uses a phishing website that looks like a real bank site to trick users into logging in, then steals their credentials.

Typical Requirement to Prevent:

- "The system shall enforce multi-factor authentication for all user logins."

T – Tampering with Data

What is Tampering?
When someone changes data without permission.

Example:

- Modifying a software file to insert malicious code.
- Changing the amount in a bank transfer during transmission.

Analogy:

Like altering a signed document to change the contract terms.

Real-world Scenario:

- Hackers inject malicious scripts into a website (e.g., form hijacking in e-commerce sites).

Typical Requirement to Prevent:

- “All transmitted data shall be encrypted and validated upon receipt.”

R – Repudiation

What is Repudiation?
When a user **denies** having performed an action, and there's **no proof** to say otherwise.

Example:

- A user deletes records and claims they didn't do it.

Analogy:

Imagine someone claiming, "I didn't send that email," and you can't prove otherwise.

Real-world Scenario:

- A system with no logging mechanism allows users to delete files without tracking who did it.

Typical Requirement to Prevent:

- "The system shall maintain detailed and tamper-proof logs of all user actions."

I – Information Disclosure

What is Information Disclosure?
When sensitive information is exposed to someone who shouldn't have access to it.

Example:

- A website mistakenly shows other users' account info.

Analogy:

Like a bank teller shouting out your account balance in public.

Real-world Scenario:

- A developer forgets to turn off debug mode, and sensitive data is shown in browser error messages.

Typical Requirement to Prevent:

- "The system shall ensure access controls are applied to all confidential data."

D – Denial of Service (DoS)

What is Denial of Service?
When someone makes a system **unavailable** to others by overwhelming it.

Example:

- A flood of traffic crashes a website so real users can't access it.

Analogy:

Like filling up all the seats in a theater with fake bookings so real people can't attend.

Real-world Scenario:

- Bot attacks on an online store during a sale to block other buyers.

Typical Requirement to Prevent:

- "The system shall detect and limit excessive requests from a single source."

E – Elevation of Privilege

What is Elevation of Privilege?
When someone gains more access or power than they're supposed to have.

Example:

- A regular user finds a flaw and becomes an admin.

Analogy:

Imagine a junior staff member finding the keys to the CEO's office and taking over.

Real-world Scenario:

- Exploiting a bug to bypass role checks and access restricted parts of the system.

Typical Requirement to Prevent:

- "The system shall enforce role-based access control for all operations."

STRIDE Summary Table

Threat	Description	Example	Requirement
Spoofing	Pretending to be someone else	Fake login credentials	Use two-factor authentication
Tampering	Changing data or files	Modify code during update	Use encryption and validation
Repudiation	Denying actions without proof	Delete logs, deny wrongdoing	Use audit logs
Info Disclosure	Exposing sensitive info	Leak user data in error	Apply access control
DoS	Overloading systems	Attack to crash website	Use rate-limiting
EoP	Gaining extra access	Regular user becomes admin	Role-based access control

Security-Critical Requirements Example

Imagine a mobile banking app:

- **Threat**: Credential theft
- **Requirement**: “The system shall enforce biometric login for all high-value transactions.”

Good security requirements must be **clear, testable, and directly address identified threats.**

Tool Support for Requirements Engineering

As systems and requirements grow in number and complexity, manual tracking becomes inefficient and risky.
Tools offer support for:

- Requirement gathering
- Traceability
- Change management
- Consistency checking

Categories of Requirements Tools

1. **Requirements Management Tools**: e.g., IBM DOORS, Jama Connect
2. **Modeling Tools**: e.g., Enterprise Architect
3. **Traceability Tools**: e.g., Reqtify
4. **Formal Verification Tools**: e.g., Alloy, SPIN

- Each tool specializes in a phase of the requirements lifecycle—from elicitation to validation.

Key Benefits of Tools

Using tools can help:

- Ensure **requirements traceability** (from user needs to code)
- Avoid **duplication or conflicts**
- Manage **requirements evolution** over time
- Facilitate **team collaboration**
- Enable **regulatory compliance** in safety/security-critical projects

Example Tool Workflow

A typical workflow could look like this:

1. Define requirements in IBM DOORS
2. Model interactions in Enterprise Architect
3. Link tests in JIRA
4. Maintain traceability with Reqtify
   This ensures consistency and confidence that all requirements are being fulfilled.

Open Research Problems in Requirements Engineering

Today's systems are **adaptive**, **intelligent**, and operate in **real-time** environments. Static requirements documents cannot capture their fluid behavior.

This evolution opens up several research problems that scholars and engineers are actively working on.

Major Open Research Problems

- **Dynamic Requirements in Agile**
  How to trace and validate rapidly evolving requirements?
- **Ambiguity in Natural Language**
  Can we auto-detect vague terms like “fast,” “secure,” or “user-friendly”?
- **AI and Learning Systems**
  How do we define “correct behavior” in systems that evolve?
- **Ethics and Responsibility**
  How do we encode social values (e.g., fairness, explainability)?
- **Automation in Requirements Elicitation**
  Can AI help extract requirements from user stories, emails, or transcripts?

Future Directions and Innovations

- **NLP** for automated requirement classification
- **Ontologies** for domain-specific vocabularies
- **Crowdsourcing** to collect public requirements
- **Digital Twins** to simulate and validate requirements in real-time

Summary

We’ve explored:

- How **hazard analysis** supports safety in critical systems
- How **threat modeling** anticipates malicious attacks
- How **tools** make requirement management scalable
- The **future** of requirements engineering in AI, ethics, and automation

As software continues to shape our world, the way we **define, manage, and validate requirements** must evolve too.